

Chapter 7 – Advanced SQL

Part 2: Recursion and Division

Databases lectures

Dr Kai Höfig



Named queries using **with**

- ◆ **Named query:** query expression that can be referenced more than once in the subsequent query (common table expression, CTE)
- ◆ Notation

```
with query_name [(column_list)] as (query_expression)
```

- ◆ Query using **with**

```
with order_list (cid,iid,price,quantity) as (  
    select orders.customer_id,  
           order_items.item_id,  
           order_items.list_price*(1-order_items.discount),  
           order_items.quantity  
    from   orders join  
           order_items on (orders.order_id=order_items.order_id)  
)  
select * from order_list
```



Reason for using **with**

- ◆ Queries become locally reusable and **with** can help to give a query a better structure and increase readability.

- ◆ Query:

```
with order_values(orderid, value) as(  
    select order_id, SUM(quantity*(list_price*(1 - discount)))  
    from order_items  
    group by order_id  
)
```

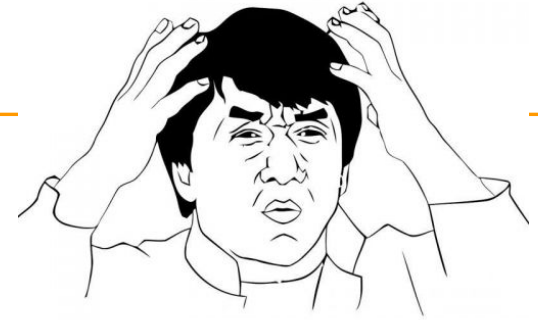
```
average_order_value(value) as (  
    select AVG(value) from order_values  
)
```

```
select *  
from order_values  
where value between (select value*0.9 from average_order_value)  
and (select value*1.1 from average_order_value)
```

What is the meaning?
How would that look
like without **with**?



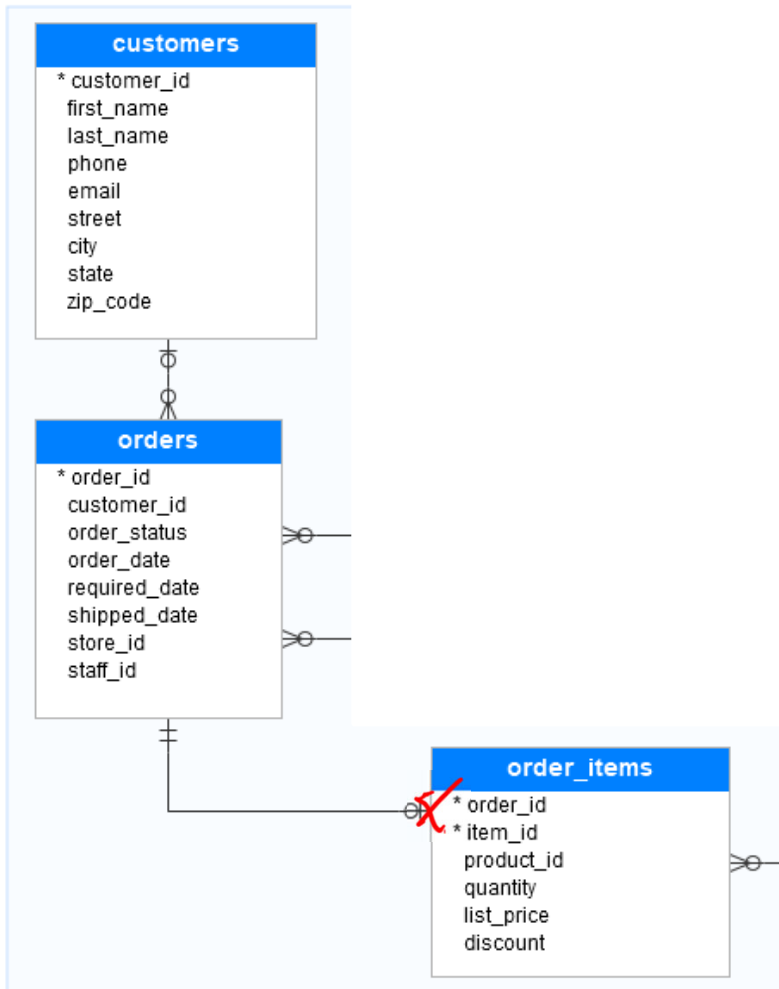
answer: it looks not so nice!



```
select *
from (select order_id, SUM(quantity*(list_price*(1 - discount))) as value
      from order_items
      group by order_id) as order_values
where value between (select avg(order_values.value)*0.9
                    from (select order_id, SUM(quantity*(list_price*(
                        from order_items
                        group by order_id) as order_values)
                    and (select avg(order_values.value)*1.1
                        from (select order_id, SUM(quantity*(list_price*(
                        from order_items
                        group by order_id) as order_values));
```



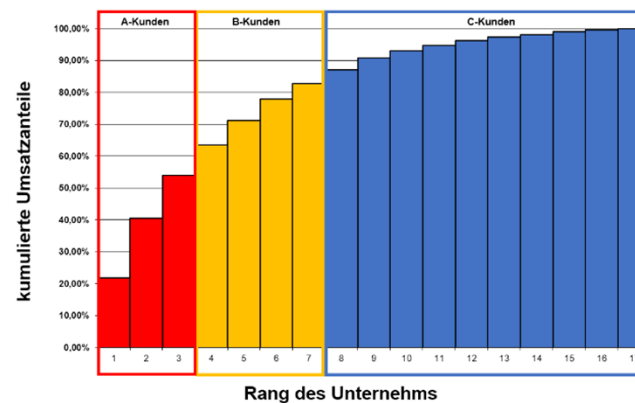
Interactive exercise: ABC analysis



An ABC analysis separates important information from unimportant information by grouping, for example:

The best customers, who make up 80% of the sales (A customers), those who make up the next 15% of the sales (B customers) and the residual 5% (C customers)

How can you specify, for each customer, whether this is an A, B, or C customer?



Fahrplanauskunft mit Echtzeitinfo und Routing von Haustür zu Haustür unter www.bayern-fahrplan.de bzw. für Smartphone m.bayern-fahrplan.de.

Brechpunkt aller Stadtverkehrslinien ist die Haltestelle Stadtmitte. Dort wechselt die Liniennummer. Alle Busse von Stadtmitte A/B und D fahren weiter zum Bhf.

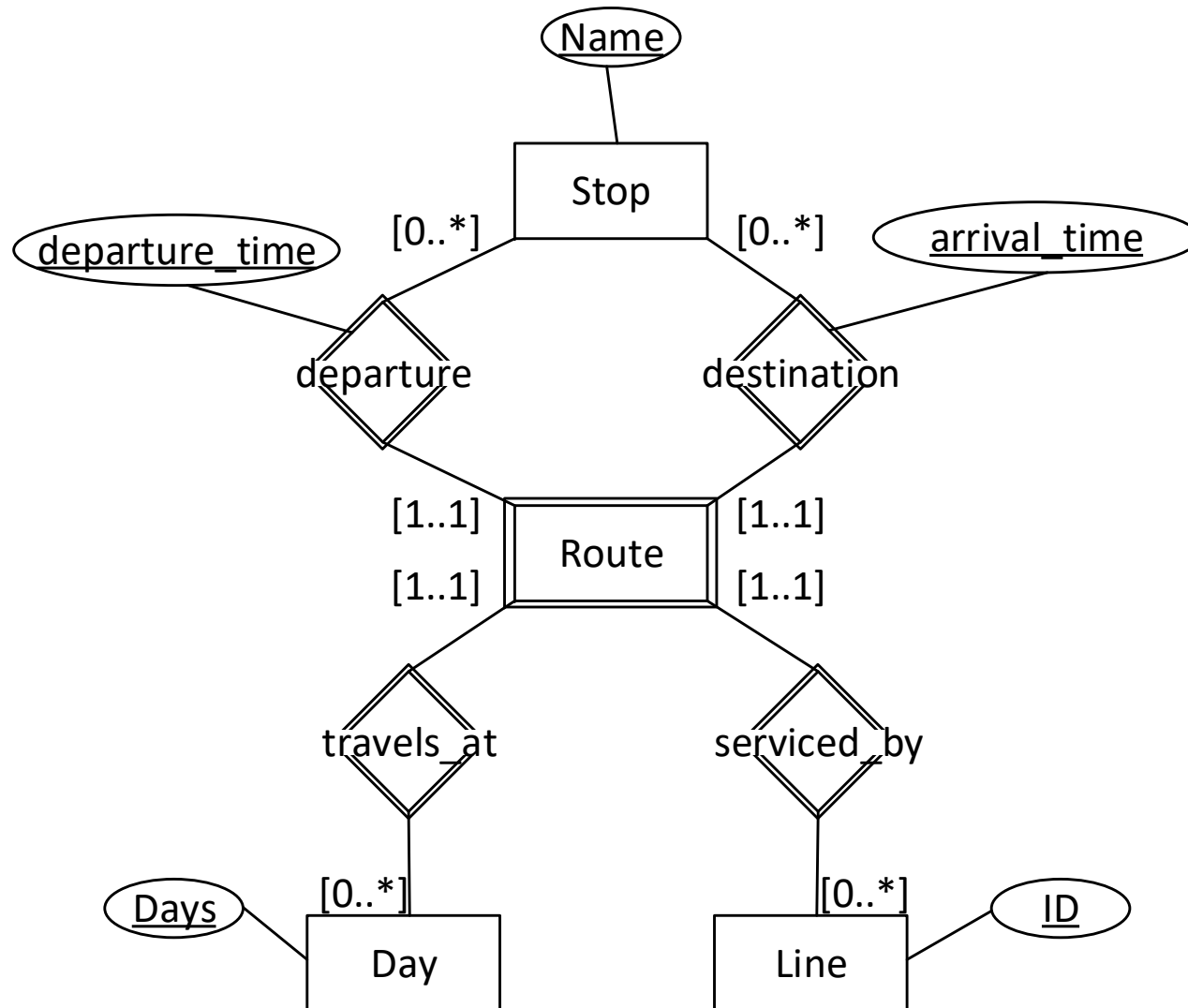




Exercise – How would a ER diagram look like?



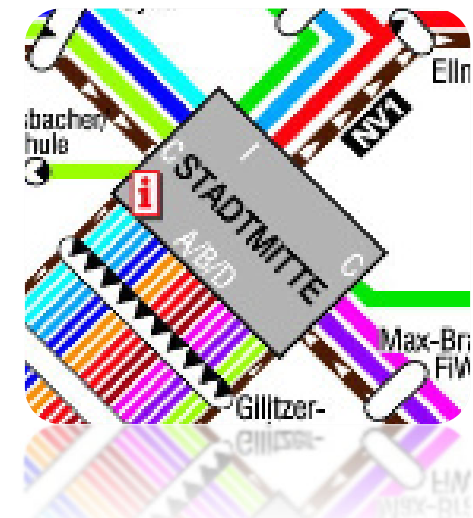
Exercise – How would a ER diagram look like?





Motivation for recursive queries

- ◆ Typical application
 - Bill of material queries (e.g. system, subsystem, subsystems...)
 - Computing the transitive closure (e.g. flight connections, bus connections)
 - Any tree-based data structure
 - etc..
- ◆ Example:
 - Bus connections in Rosenheim
 - Question: what are all the places we can go to by bus from “Stadtmitte”?



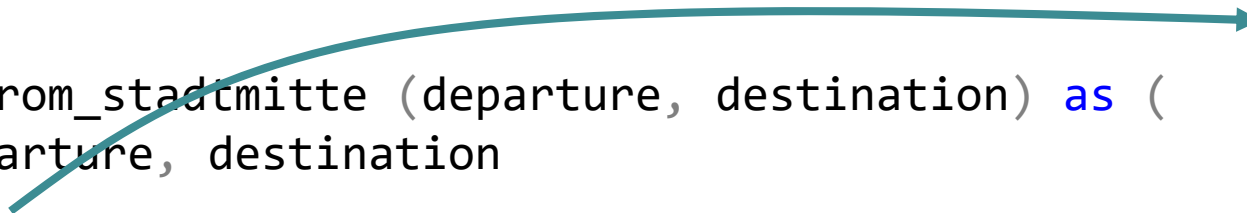
departure	destination	line	dep_time	arr_time	day
Stadtmitte	Max-Bram-Pl./FiWaG	10	08:00	08:04	Mo-Fr
Stadtmitte	Ignaz Günther Gymnasium	1	08:00	08:05	Mo-Fr



Unrolling a recursive data structure using `with` – One stop

- ◆ First, we see where we can get with one stop

```
with bus_from_stadtmitte (departure, destination) as (  
  select departure, destination  
  from   bus  
  where  departure='Stadtmitte'  
)  
select * from bus_from_stadtmitte;
```



Datasource:
Bus



Unrolling a recursive data structure using `with` – Two stops

- ◆ Then, we see where we can get with two stops

Datasource:
Bus

```
with
bus_from_stadtmitte_1 (departure, destination) as (
select departure, destination
from bus
where departure='Stadtmitte'
),
bus_from_stadtmitte_2 (departure, destination) as (
select bus_from_stadtmitte_1.departure, bus.destination
from bus_from_stadtmitte_1, bus
where bus_from_stadtmitte_1.destination=bus.departure
)
```

```
select * from bus_from_stadtmitte_1
union
select * from bus_from_stadtmitte_2;
```



Unrolling a recursive data structure using `with` – Three stops

- ◆ Then, we see where we can get with three stops

```
with
bus_from_stadtmitte_1 (departure, destination) as (
select departure, destination
from bus
where departure='Stadtmitte'
),
bus_from_stadtmitte_2 (departure, destination) as (
select bus_from_stadtmitte_1.departure, bus.destination
from bus_from_stadtmitte_1, bus
where bus_from_stadtmitte_1.destination=bus.departure
),
bus_from_stadtmitte_3 (departure, destination) as (
select bus_from_stadtmitte_2.departure, bus.destination
from bus_from_stadtmitte_2, bus
where bus_from_stadtmitte_2.destination=bus.departure
)
```

```
select * from bus_from_stadtmitte_1
union
select * from bus_from_stadtmitte_2
union
select * from bus_from_stadtmitte_3;
```

Datasource:
Bus



Using recursion

- Using recursion, we can now unroll that data structure*.

```
with
bus_from_stadtmitte (departure,destination) as (
select departure, destination
from bus
where departure='Stadtmitte'

union all

select bus_from_stadtmitte.departure, bus.destination
from bus_from_stadtmitte, bus
where bus_from_stadtmitte.destination=bus.departure
)

select * from bus_from_stadtmitte;
```

Datasource:
Bus

** we will deal with circles later*



Recursive queries – Schematic

```
with recursion_table as (
```

```
select ...  
from table  
where ...
```

Initialisation

```
union all
```

Recursive part

```
select ...  
from table, recursion_table  
where recursion_condition
```

Recursion step

```
)
```

```
query_expression
```

Non-recursive part



Step-by-step unrolling of the recursion

```
select departure, destination
from bus
where departure='Stadtmitte'
```

1

departure	destination
Stadtmitte	Max-Bram-Pl./FiWaG
Stadtmitte	Ignaz Günther Gymnasium

```
select bus_from_stadtmitte.departure, bus.destination
from bus_from_stadtmitte, bus
where bus_from_stadtmitte.destination=bus.departure
```

2 3 4 ...

departure	destination
Stadtmitte	Hohenzollernstr.
Stadtmitte	Ignaz Günther Gymnasium
Stadtmitte	Innsbrucker Str.
Stadtmitte	Max-Bram-Pl./FiWaG

departure	destination
Stadtmitte	Berufsschule
Stadtmitte	Giessenbachstr.
Stadtmitte	Hohenzollernstr.
Stadtmitte	Ignaz Günther Gymnasium
Stadtmitte	Innsbrucker Str.
Stadtmitte	Max-Bram-Pl./FiWaG

departure	destination
Stadtmitte	Berufsschule
Stadtmitte	Giessenbachstr.
Stadtmitte	Hohenzollernstr.
Stadtmitte	Ignaz Günther Gymnasium
Stadtmitte	Innsbrucker Str.
Stadtmitte	Königsseestr./Unter den Eichen
Stadtmitte	Marienbergerstr.
Stadtmitte	Max-Bram-Pl./FiWaG



Recursive calculations

♦ Example: Number of stops

```
with
bus_from_stadtmitte (departure,destination,stops) as (
select departure, destination,1
from bus
where departure='Stadtmitte'

union all

select bus_from_stadtmitte.departure, bus.destination, stops+1
from bus_from_stadtmitte, bus
where bus_from_stadtmitte.destination=bus.departure
)
select * from bus_from_stadtmitte
```

Initialization

Recursive step



Computability of recursive queries

- ◆ Computability (= finiteness of the calculation) is an important requirement for the query language
- ◆ Problem: cycles with recursion
- ◆ 3 ways to handle it in SQL
 - 1) Design the query properly
 - 2) Limiting the recursion level using `OPTION (MAXRECURSION <X>)`
 - 3) Cycle detection
(defined in the standard since SQL:2003, not yet implemented in any DBMS)





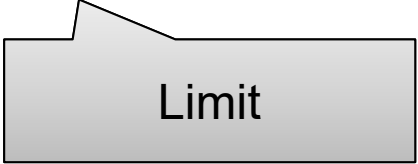
Example for limiting the recursion depth

- ◆ Example: max. 4 changes

```
with
bus_from_stadtmitte (departure,destination,stops) as (
select departure, destination,1
from   bus
where  departure='Stadtmitte'

union all

select bus_from_stadtmitte.departure, bus.destination, stops+1
from   bus_from_stadtmitte, bus
where  bus_from_stadtmitte.destination=bus.departure
)
select * from bus_from_stadtmitte where stops <5
```



Limit



Exercise: Manual circle detection



Exercise: Manual circle detection

```
with
bus_from_stadtmitte (departure,destination,way) as (
select departure, destination, cast(departure+'->'+destination as varchar(max))
from bus
where departure='Stadtmitte'

union all

select bus_from_stadtmitte.departure, bus.destination, way+'->'+bus.destination
from bus_from_stadtmitte, bus
where bus_from_stadtmitte.destination=bus.departure and
      way not like '%'+bus.destination+'%'
)

select * from bus_from_stadtmitte;
```

Limit

Division

1-16 of over 7,000 results for "laptop"

More-sustainable Products☐ Climate Pledge Friendly**Department**

Laptop Computers

Traditional Laptop Computers

2 in 1 Laptop Computers

Customer Reviews

★★★★★ & Up

★★★★☆ & Up

★★★☆☆ & Up

★★☆☆☆ & Up

Brands☐ HP☐ Lenovo☐ ASUS☐ Dell☐ acer☐ msi☐ Microsoft

▾ See more

Laptop Price

Under \$500

\$500 to \$600

\$600 to \$700

\$700 to \$800

\$800 to \$1,000

Results

Sponsored ⓘ

ASUS Chromebook CX1, 15.6" Full HD NanoEdge Display, Storage, 4GB RAM, Spill-Resistant Keyboard, Chrome OS,

★★★★☆ ▾ 4

\$178⁷⁸Delivery **Tue, Nov 28**

Ships to Germany



Sponsored ⓘ

Lenovo IdeaPad, 20GB RAM, 1TB SSD, AMD Dual-core Pro Battery Life Up to 9.5Hr, HDMI, SD Card Reader, Windows

★★★★☆ ▾ 702

\$449⁹⁹

Save 10% with coupon

Delivery **Tue, Nov 28**

Ships to Germany

Options: 7 capacities

Overall Pick ⓘ

**HP 14 Laptop, Intel Celeron N4020, 4 GB RAM, 64 GB Storage, Home, Thin & Portable, 4K Graphics, One Year of Microsoft**

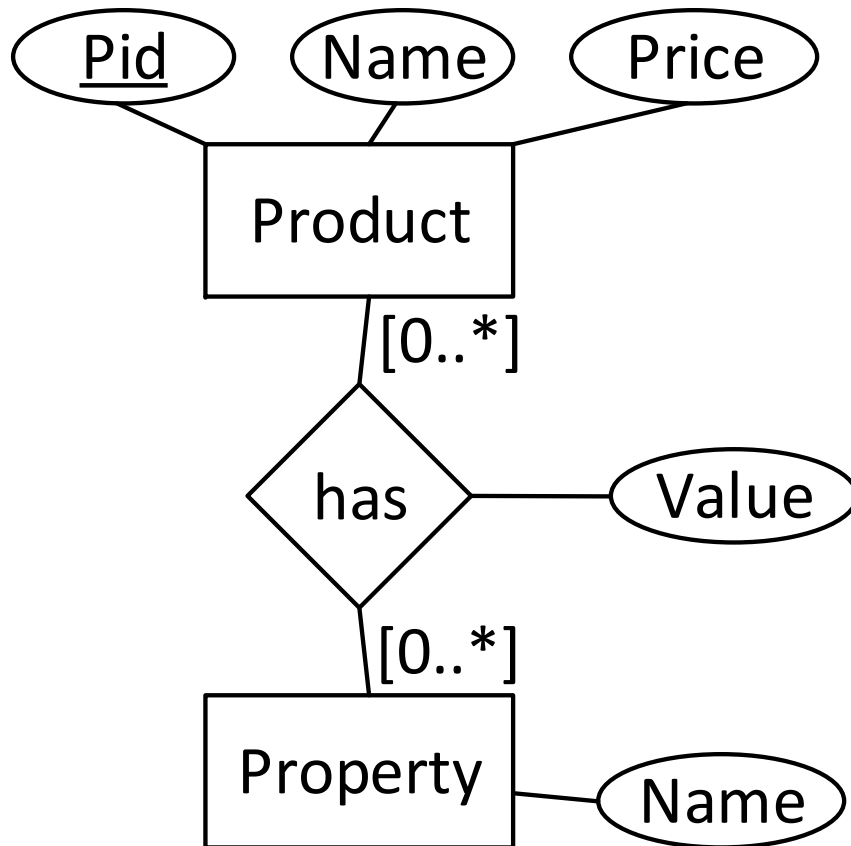
★★★★☆ ▾ 500



Exercise: ER Diagram



Exercise: ER Diagram

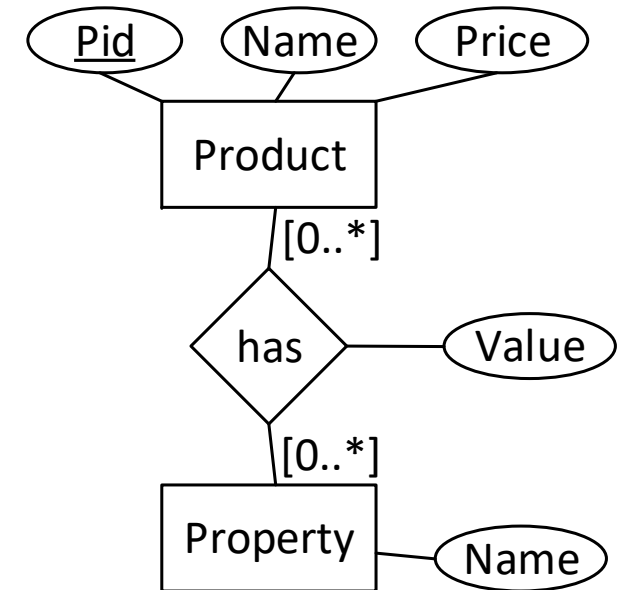




Division – Motivating example: A filter by multiple attributes

- “I need all laptops from Lenovo with Intel CPUs and numeric keypad”

```
select * from products
join has_property on
(products.pid=has_property.pid)
```



pid	name		price	pid	name		value
2323	Lenovo	IdeaPad	449.99	2323	CPU		Intel
2323	Lenovo	IdeaPad	449.99	2323	Vendor		Lenovo
2323	Lenovo	IdeaPad	449.99	2323	has keypad		NULL
2355	Lenovo	V15 Laptop	492.99	2355	Vendor		Lenovo
2355	Lenovo	V15 Laptop	492.99	2355	CPU		AMD



Division of relations

- ♦ Analogy with the arithmetic operation of integer division: integer division is the inverse of (integer) multiplication. It produces the largest number for which multiplication by the divisor is less than the dividend.

$$\begin{array}{ccc} & \text{Dividend} & \\ \text{Quotient} & x = \frac{a}{b} := \{x | x \in \mathbb{N}, x \cdot b \leq a, x \text{ max.}\} & \\ & \text{Divisor} & \end{array}$$

- ♦ Similarly for relations, it is

$$r = \frac{r_1}{r_2} := \{r | r \times r_2 \subseteq r_1, r \text{ max.}\}$$



Simple example

Movies:

movie	property
Die Hard	Action
Die Hard	Christmas
They Live	Sci-Fi
They Live	Trash
They Live	Horror
Kin-dza-dza!	Sci-Fi
Kin-dza-dza!	Trash
Dead Snow	Trash
Dead Snow	Splatter



Die Hard 1988
Action, Christmas



They Live 1988
SciFi, Trash, Horror



Kin-dza-dza! 1986
SciFi, Trash



Dead Snow 2009
Trash, Splatter

Favourite_properties:

property
Sci-Fi
Trash

Favourite_movies:

movie
They Live
Kin-dza-dza!



Favourite_properties:

property
Sci-Fi
Trash

$\subseteq \text{Movies}$

Die Hard

$\not\subseteq \text{Movies}$

$$\text{Movies}_{fav} = \frac{\text{Movies}}{\text{Properties}_{fav}} := \{\text{Movies}_{fav} | \text{Movies}_{fav} \times \text{Properties}_{fav} \subseteq \text{Movies}, \text{Movies}_{fav} \text{ max.}\}$$



Schematic implementation

$$r = \frac{r_1}{r_2} := \{r \mid r \times r_2 \subseteq r_1, r \text{ max.}\}$$

- ◆ Can be expressed in the relational algebra using

$$r = \frac{r_1}{r_2} = \pi_{R'}(r_1) - \pi_{R'}((\pi_{R'}(r_1) \times r_2) - r_1)$$

```
select <attributes> from R1
except
select <attributes> from (
    select <attributes>, R2.* from R1, R2
    except
    select * from R1
) as x
```

Leave non-existing property combinations and they kick-out tuples that do not fulfill all properties

(all possible property combinations
minus
all real property combinations)



Alternative implementation

```
select distinct <attributes> from R1 R1a
where not exists (
    select * from R2
    where not exists (
        select * from R1 R1b
        where R1a.<attributes> = R1b.<attributes> and
              r1b.<property>=R2.<property>
    )
)
```

- ◆ Linguistically: "Output all the elements of R1 for which there is no property which is not in the list of R2."



Implementation of the movie example

```
select movie from movies
except
select movie from (
    -- all combinations that could exist
    select movie, favourite_properties.property
    from movies, favourite_properties
    except
    -- all real combinations of properties
    select movie, property from movies
) as properties_that_do_not_exist;
```

the properties that are being filtered for are not in the select

◆ Alternatively:

```
select distinct movie from movies as m1
where not exists (
    select * from favourite_properties
    where not exists (
        select * from movies as m2
        where m1.movie=m2.movie and
              m2.property=favourite_properties.property
    )
);
```

“Output all the movies for which there is no property which is not in the list of wanted properties.”



Solution for the laptop filter

Local temporary table,
since search properties
cannot be stored
permanently in the DB

- ◆ Same concept, but some joins and some formatting of attributes is required

```
create table #myproperties(propname varchar(30), propvalue varchar(30))
insert into #myproperties values
    ('Vendor', 'Lenovo'), ('CPU', 'Intel'), ('has keypad', null)
```

properties and products
are not in the same table

```
select products.pid, name, price
from products join has_property on (products.pid=has_property.pid)
except
```

```
select pid, name, price from (
    select * from products, #myproperties
except
select products.pid, name, price, propname, propvalue
from products join has_property on
    (products.pid=has_property.pid)
```

```
)as x
```

all possible property
product combinations

the actual existing
property combinations
(again, not in the same
table)



Why is division important?

- ◆ an arbitrary number of attributes can be included in filtering. The attributes can be in a different table and the query can be extended by more attributes easily.
- ◆ The resulting elements (i.e. products) are generated by the DBMS and only the elements that fulfil the selected properties are transferred to the client.
- ◆ The client does not need to further process that data nor must discard data.

